



Faculdade De Engenharia Da Universidade Do Porto

Project 1

Performance evaluation of Single and Multi core Implementation

Parallel and Distributed Computing

T10 - G16

-Aléssia Jorge (up202109242@up.pt)

-Luís Freitas (up201905767@up.pt)

-Mariana Conde (up202108824@up.pt)

21. March. 2024

Project description

This project involves implementing and comparing different matrix multiplication algorithms using C++ and another programming language, evaluating their performance on both single-core and multicore architectures. The main objective is to study the effect of memory access patterns on processor performance when handling large datasets, using PAPI for performance analysis and OpenMP for parallelization. To achieve this, we implemented three different matrix multiplication approaches and measured their execution time across various matrix sizes.

This report is organized as follows:

- **Section 1** - Description of the implemented algorithms
- **Section 2** - Performance metrics used
- **Section 3** - Obtained results and a detailed analysis of the performance of the different algorithm versions
- **Section 4** - Conclusions we got from the project

1. Algorithms Explanation

In this project, parallel implementation was used, allowing the program to be tested in both C++ and Java (our chosen language) to compare potential differences in execution time and memory access patterns between the two programming languages.

To test each performance, three different matrix multiplication techniques were implemented. And we also implemented two parallel versions for the second implementation. These techniques were:

1. Simple Matrix Multiplication (already provided in the project's initial code)
2. Line Matrix Multiplication
3. Block Matrix Multiplication

1.1. Simple Matrix Multiplication

This algorithm at its core performs a simple matrix multiplication of two square matrices. This function uses a triple nested loop in order to multiply said matrices. The outer loop iterates over the rows of matrix A, the middle loop iterates over the columns of matrix B and the innermost loop performs the dot product of the two previous results (matrix A row and matrix B column) and stores them in a new Matrix C.

The time complexity of this matrix multiplication algorithm is $O(n^3)$ where n is the dimension of the square matrices. This is a direct result from the usage of a three nested loop, each iterating n times.

1.2. Line Matrix Multiplication

This algorithm also performs matrix multiplication but modifies the order of operations, this function multiplies an element from the first matrix by the corresponding line of the second matrix.

1.3. Block Matrix Multiplication

This algorithm improves cache utilization and reduces memory access latency, dividing the matrices into smaller submatrices (blocks) and computing their products separately. The function is performed block by block, following the same multiplication rules, but with a limited scope of these smaller blocks.

1.4. Parallel Implementation

This version introduces parallelism of the second implementations of the matrix product, using OpenMP to distribute the workload across multiple CPU cores. Uses `#pragma omp parallel` for to enable multiple threads to perform multiplication simultaneously

1.4.1 Parallel 1

The first Parallel Solution divides the workload by distributing rows of the resulting matrix across multiple threads. Each thread computes the multiplication of a subset of rows in parallel. This approach aims to exploit multicore architectures to enhance overall throughput. Here is a piece of algorithm:

```
#pragma omp parallel for private(i, j, k)
for(i = 0; i < m_ar; i++)
{for(k = 0; k < m_ar; k++)
    {for(j = 0; j < m_br; j++)
        {phc[i*m_ar + j] += pha[i*m_ar + k] * phb[k*m_br + j];}
    }
}
Time2 = omp_get_wtime();
sprintf(st, "Time: %3.3f seconds\n", (double)(Time2 - Time1));
```

1.4.2. Parallel 2

The second solution parallelizes the computation by distributing both rows and columns of the resulting matrix across threads. Each thread is responsible for computing a portion of the resulting matrix. By parallelizing both row and column computations, this approach seeks to further leverage multicore processing capabilities. Here is a piece of algorithm:

```

#pragma omp parallel for private(j, k) shared(pha, phb, phc)
for(i = 0; i < m_ar; i++) {
    for(k = 0; k < m_ar; k++) {
        #pragma omp
        for(j = 0; j < m_br; j++) {
            phc[i * m_br + j] += pha[i * m_ar + k] * phb[k * m_br + j];
        }
    }
}

Time2 = omp_get_wtime();

```

2. Performance Metrics

To evaluate the different performance of the algorithms for the code written in C++, the Performance API (PAPI) was used, as demanded by the worksheet. This API allows the computer to measure the time spent on each algorithm, essential for the determination of the optimal algorithm in terms of execution time. Simultaneously, the API also measures cache misses. In our work, we analyzed the Level 1 and Level 2 data cache misses.

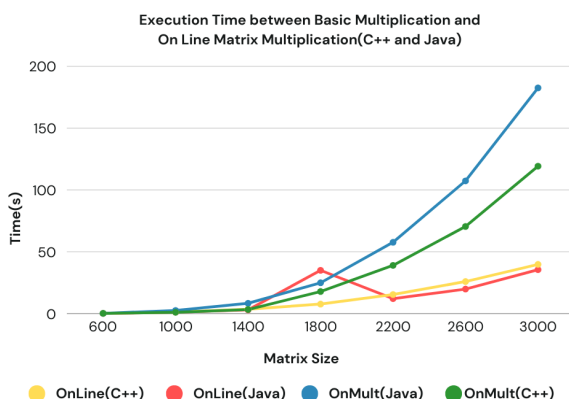
The testing environment for most tests was a computer with an Intel Core i7-9700 processor with 3 GHz CPU clock speed and 16 GB of RAM. However, for the first run of tests, the computer used had a single clock speed of 4.7 GHz, meaning that the cycles the CPU could execute per second was higher than the previous one. Thus allowing us to reach the conclusion that a higher CPU clock speed also affects the performance of these algorithms, since the instructions are processed faster, a better and faster performance is achieved.

In terms of compiling, java was compiled as default, contrary to the C++ implementation that was compiled with 3 extra flags, a flag dedicated to the Performance API (*-lpapi*), another dedicated to the use of OpenMP (*-fopenmp*) and finally an optimization flag (*-O2*) as suggested.

3. Results and Analysis

In this section, we present and analyze the performance results obtained from the different matrix multiplication algorithms. The evaluation focuses on execution time of different matrix sizes, the average cache misses L1, the efficiency and speedup.

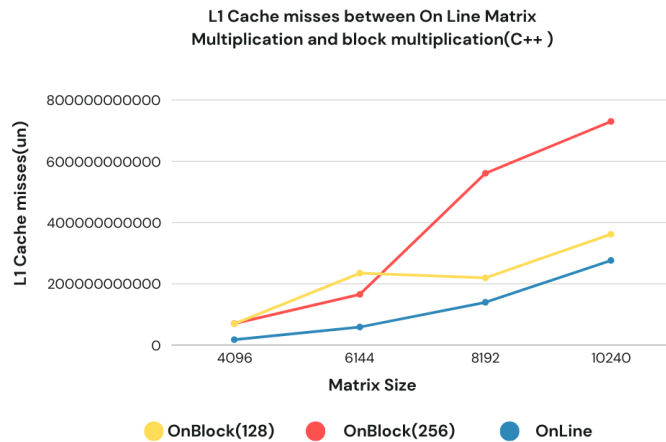
1. Execution time comparison between C++ and Java Implementation



The graph shows that the OnMult implementation has a significantly higher execution time compared to OnLine as the matrix size increases. This indicates that the line-by-line approach can be more efficient, especially in C++, where the performance difference becomes more

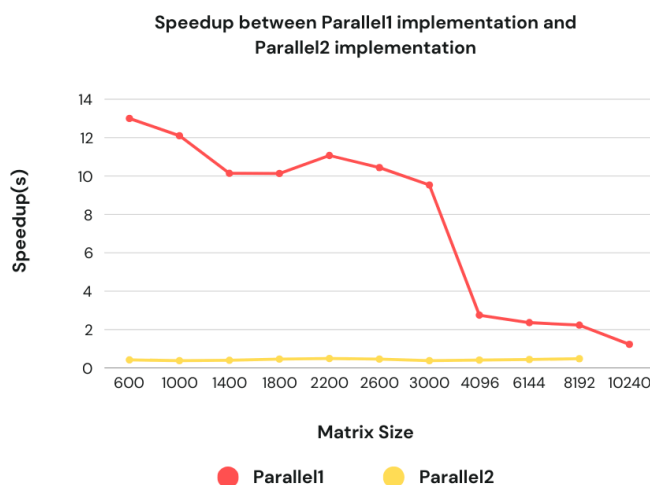
evident for larger matrices. The Java implementation shows lower execution times for intermediate sizes, but suffers from higher growth for large matrices.

2. Cache performance between OnLine and Block multiplication



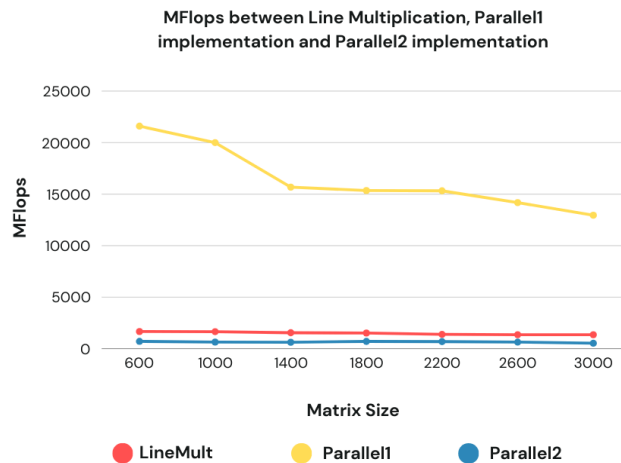
The L1 cache miss analysis shows that OnLine multiplication generates significantly fewer cache misses compared to OnBlock(128) and OnBlock(256). However, for larger matrix sizes, OnBlock(256) has a much larger increase in cache misses, that shows that larger blocks may not fit well in the cache hierarchy, making them less efficient. OnBlock(128) maintains a relatively low number of misses, indicating better cache utilization for medium-sized matrices.

3. Speedup comparison between the two parallel versions



The comparison between Parallel1 and Parallel2 shows that Parallel1 initially achieves a high speedup, reaching high values for small matrices. However, as the matrix size increases, Parallel1 rapidly loses efficiency, especially after 4096. On the other hand, Parallel2 maintains a consistent but ineffective performance, indicating that its implementation does not efficiently leverage parallelism.

4. MFlops comparison between OnLine multiplication and the two parallel versions



The graph compares the performance of three matrix multiplication implementations (LineMult, Parallel1, and Parallel2) in terms of MFLOPS across different matrix sizes. Parallel1 significantly outperforms the other two, reaching over 20,000 MFLOPS for smaller matrices but gradually declining as the matrix size increases, while LineMult and Parallel2 maintain consistently low performance, with Parallel2 being the least efficient. The results show that Parallel1 benefits from parallelization more than Parallel2.

4. Conclusion

To conclude, while making the project we notice that the differences between Java and C++ become more evident as matrix size increases with C++ performing better because of better memory management and optimized compilation. Regarding algorithm performance, OnMult is the slowest while OnMultLine and OnMultBlock improve execution time and cache efficiency, with OnMultBlock performing the best due to block reuse. Parallelization further enhances performance by reducing time complexity to $O(n^3/p)$, improving both execution speed and cache usage. Overall, this analysis highlights the impact of memory management and algorithm choice on execution time and efficiency.

Annexes

- OnMult

Matrix Size	Elapsed Time (s)	L1 DCM	L2 DCM	Matrix Size	Averages	
600 x 600	0.183	244757019	39865755	600 x 600	Elapsed Time	0.185
	0.185	244784984	37544336		L1 DCM	244771563.7
	0.187	244772688	37260705		L2 DCM	38223598.67
1000 x 1000	1.116	1227995120	310488143	1000 x 1000	Elapsed Time	1.058666667
	0.96	1228076277	196687967		L1 DCM	1224258202
	1.1	1216703210	209225468		L2 DCM	238800526
1400 x 1400	2.985	3433856118	1372535611	1400 x 1400	Elapsed Time	3.264
	3.14	3501539298	519318898		L1 DCM	3446193864
	3.667	3403186175	592653142		L2 DCM	828169217
1800 x 1800	17.822	90779456840	7691141868	1800 x 1800	Elapsed Time	17.83833333
	17.675	9094677153	3531767646		L1 DCM	36321456017
	18.018	9090234057	5173638796		L2 DCM	5465516103
2200 x 2200	38.563	17629676460	22666415490	2200 x 2200	Elapsed Time	39.23166667
	39.034	17631335718	18933131050		L1 DCM	17634113704
	40.098	17641328934	20623581544		L2 DCM	20741042695
2600 x 2600	68.347	30912084895	49997800236	2600 x 2600	Elapsed Time	70.476
	71.196	30908193534	50181642033		L1 DCM	30906123853
	71.885	30898093129	50299240641		L2 DCM	50159560970
3000 x 3000	117.295	50292014705	95448398754	3000 x 3000	Elapsed Time	119.259
	119.496	50299309331	1.00127E+11		L1 DCM	50304544755
	120.986	50322310230	97346916210		L2 DCM	97640749944

- OnLine

Matrix Size	Elapsed Time (s)	L1 DCM	L2 DCM	Matrix Size	Averages	
600 x 600	0.585	27098823	59612541	600 x 600	Elapsed Time	0.262666667
	0.101	27108537	58290405		L1 DCM	27106837
	0.102	27113151	56899856		L2 DCM	58267600.67
1000 x 1000	2.716	125737727	276936952	1000 x 1000	Elapsed Time	1.217
	0.464	125731060	267797620		L1 DCM	125734715.7
	0.471	125735360	262124650		L2 DCM	268953074
1400 x 1400	7.593	352511500	774128182	1400 x 1400	Elapsed Time	3.554333333
	1.523	346111377	717017336		L1 DCM	348242554.3
	1.547	346104786	707716300		L2 DCM	732953939.3
1800 x 1800	16.268	762513976	1660660941	1800 x 1800	Elapsed Time	7.705666667
	3.464	745220466	1443867055		L1 DCM	750980137.3
	3.385	745205970	1455282702		L2 DCM	1519936899
2200 x 2200	30.96	2108630654	2992061586	2200 x 2200	Elapsed Time	15.39233333
	7.603	2071275524	2417864540		L1 DCM	2083804775
	7.614	2071508148	2455133288		L2 DCM	2621686471
2600 x 2600	49.469	4394881423	4934426970	2600 x 2600	Elapsed Time	25.91966667
	14.043	4412010518	3872673476		L1 DCM	3082689313
	14.247	441175998	3919788772		L2 DCM	4242296406
3000 x 3000	75.906	6757186552	7586815117	3000 x 3000	Elapsed Time	39.78533333
	21.701	6780696396	5834375078		L1 DCM	6772712022
	21.749	6780253118	5921939388		L2 DCM	6447709861
4096 x 4096	198.404	17792034084	19356204072	4096 x 4096	Elapsed Time	106.438
	60.346	17497212914	15155793872		L1 DCM	17589468927
	60.564	17479159783	14550201267		L2 DCM	16354066404
6144 x 6144	677.78	58305075521	65376859042	6144 x 6144	Elapsed Time	361.6266667
	203.602	58967081975	49725474251		L1 DCM	58734683734
	203.498	58931893707	49301549711		L2 DCM	54801294335
8192 x 8192	1672.608	1.38707E+11	1.54713E+11	8192 x 8192	Elapsed Time	890.8913333
	500.504	1.39842E+11	1.19526E+11		L1 DCM	1.39444E+11
	499.562	1.39784E+11	1.1801E+11		L2 DCM	1.3075E+11
10240 x 10240	3245.884	2.82946E+11	3.01393E+11	10240 x 10240	Elapsed Time	1717.634667
	952.638	2.72557E+11	2.2717E+11		L1 DCM	2.76042E+11
	954.382	2.72622E+11	2.27365E+11		L2 DCM	2.51976E+11

- OnBlock

Block Size	Matrix Size	Elapsed Time (s)	L1 DCM	L2 DCM
128	4096 x 4096	89.589	70006356431	1.08089E+11
		109.032	70006669827	1.07714E+11
		106.883	70015303082	1.21439E+11
	6144 x 6144	298.703	2.36277E+11	3.54895E+11
		358.961	23631452301	3.7921E+11
		363.641	2.36308E+11	3.45278E+11
	8192 x 8192	741.31	5.60052E+11	9.05884E+11
		939.21	5.60752E+11	8.49995E+11
		939.21	5.60752E+11	8.49995E+11
	10240 x 10240	1373.444	1.09415E+12	1.75304E+12
		1717.272	1.0951E+12	1.72194E+12
		Killed	-	-
256	4096 x 4096	91.961	69491221639	90718704412
		118.927	69517558982	1.0342E+11
		116.828	69520187867	1.10658E+11
	6144 x 6144	295.786	2.3441E+11	3.82069E+11
		406.877	2.34703E+11	3.41075E+11
		406.242	2.34703E+11	3.41238E+11
	8192 x 8192	1512.65	6.57852E+11	9.01193E+11
		-	-	-
		-	-	-
	10240 x 10240	1590.803	1.0849E+12	1.60147E+12
		-	-	-
		-	-	-
512	4096 x 4096	148.89	71791414692	1.08341E+11
		760.137	70098124849	74800837493
		746.432	70109593492	74640937474
	6144 x 6144	288.299	2.33491E+11	3.8599E+11
		418.619	2.33932E+11	3.51768E+11
		495.663	2.35982E+11	3.48524E+11
	8192 x 8192	-	-	-
		-	-	-
		-	-	-
	10240 x 10240	1351.889	1.08203E+12	1.87672E+12
		-	-	-
		-	-	-

Block Size	Matrix Size	Averages	
128	4096 x 4096	Elapsed Time	101.834667
		L1 DCM	7.0009E+10
		L2 DCM	1.1241E+11
	6144 x 6144	Elapsed Time	340.435
		L1 DCM	1.6541E+11
		L2 DCM	3.5979E+11
	8192 x 8192	Elapsed Time	873.243333
		L1 DCM	5.6052E+11
		L2 DCM	8.6862E+11
	10240 x 10240	Elapsed Time	1545.358
		L1 DCM	1.0946E+12
		L2 DCM	1.7375E+12
256	4096 x 4096	Elapsed Time	109.238667
		L1 DCM	6.951E+10
		L2 DCM	1.016E+11
	6144 x 6144	Elapsed Time	369.635
		L1 DCM	2.3461E+11
		L2 DCM	3.5479E+11
	8192 x 8192	Elapsed Time	1512.65
		L1 DCM	6.5785E+11
		L2 DCM	9.0119E+11
	10240 x 10240	Elapsed Time	1590.803
		L1 DCM	1.0849E+12
		L2 DCM	1.6015E+12
512	4096 x 4096	Elapsed Time	551.819667
		L1 DCM	7.0666E+10
		L2 DCM	8.5927E+10
	6144 x 6144	Elapsed Time	400.860333
		L1 DCM	2.3447E+11
		L2 DCM	3.6209E+11
	8192 x 8192	Elapsed Time	-
		L1 DCM	-
		L2 DCM	-
	10240 x 10240	Elapsed Time	1351.889
		L1 DCM	1.082E+12
		L2 DCM	1.8767E+12

- Parallel1

Matrix Size	Elapsed Time (s)	L1 DCM	L2 DCM
600 x 600	0.015	1427163	2983895
	0.037	3391929	7244175
	0.036	3396424	7168779
1000 x 1000	0.066	4102496	8969277
	0.123	15717605	32782427
	0.122	15719626	32978480
1400 x 1400	0.195	22427205	25219797
	0.438	43454232	84129017
	0.443	43453123	83648730
1800 x 1800	0.439	68583646	51177950
	0.888	93562283	170377853
	0.977	93555460	172644925
2200 x 2200	0.803	125949677	94350540
	1.75	258287930	302805398
	1.642	258332210	306822548
2600 x 2600	1.367	208090436	154205371
	2.945	5493919915	497081613
	3.138	549435035	496445049
3000 x 3000	2.047	329362395	259498771
	5.701	845097270	746222292
	4.786	845084655	762722226
4096 x 4096	7.337	935014823	797398127
	54.702	2177325873	1581230578
	54.088	2176906225	1567840831
6144 x 6144	34.732	3251403610	2540854156
	211.309	7341427966	4916427879
	213.496	7342712920	4945182761
8192 x 8192	134.897	8224593473	6991677690
	525.036	17348757316	11808508971
	535.487	17346843682	11748459916
10240 x 10240	378.054	16855674065	2080131225
	1876.533	34044615077	24399584212
	1929.189	34054410812	24580729653

Matrix Size	Averages	
600 x 600	Elapsed Time	0.029333333
	L1 DCM	2738505.333
	L2 DCM	5798949.667
1000 x 1000	Elapsed Time	0.103666667
	L1 DCM	11846575.67
	L2 DCM	24910061.33
1400 x 1400	Elapsed Time	0.358666667
	L1 DCM	36444853.33
	L2 DCM	64332514.67
1800 x 1800	Elapsed Time	0.768
	L1 DCM	85233796.33
	L2 DCM	131400242.7
2200 x 2200	Elapsed Time	1.398333333
	L1 DCM	214189939
	L2 DCM	234659495.3
2600 x 2600	Elapsed Time	2.483333333
	L1 DCM	2083815129
	L2 DCM	382577344.3
3000 x 3000	Elapsed Time	4.178
	L1 DCM	673181440
	L2 DCM	589481096.3
4096 x 4096	Elapsed Time	38.709
	L1 DCM	1763082307
	L2 DCM	1315489845
6144 x 6144	Elapsed Time	153.179
	L1 DCM	5978514832
	L2 DCM	4134154932
8192 x 8192	Elapsed Time	398.4733333
	L1 DCM	14306731490
	L2 DCM	10182882192
10240 x 10240	Elapsed Time	1394.592
	L1 DCM	28318233318
	L2 DCM	17020148363

- Parallel2

Matrix Size	Elapsed Time (s)	L1 DCM	L2 DCM
600 x 600	0.306	3392506	7476452
	0.778	3394899	7196921
	0.766	3394292	7208041
1000 x 1000	1.357	15732167	35050697
	3.483	15733773	30937816
	4.539	15741533	31717265
1400 x 1400	3.395	44407160	96775519
	11.126	43869411	85285420
	12.085	43868443	83987764
1800 x 1800	7.342	96602617	207877732
	25.224	94747125	174502693
	17.443	94534998	176261218
2200 x 2200	14.618	260710960	373761435
	38.729	263583608	334024293
	39.547	263382792	325460600
2600 x 2600	22.073	541064439	617078024
	71.385	549717777	541774946
	72.327	549587499	529008067
3000 x 3000	30.813	837926495	946444095
	138.594	845184525	834916480
	139.763	844845334	822284615
4096 x 4096	81.967	2150526503	2425279861
	346.33	2163740586	2232684105
	345.544	2164457470	2242698442
6144 x 6144	320.69	7269888502	8165900980
	1063.485	7323064842	7603338077
	1061.855	7330681194	7643646236
8192 x 8192	-	-	-
	1799.099	17214055039	19192486911
	1828.978	17214123776	19155970907
10240 x 10240	-	-	-
	-	-	-
	Killed	-	-

Matrix Size	Averages	
600 x 600	Elapsed Time	0.616666667
	L1 DCM	3393899
	L2 DCM	7293804.67
1000 x 1000	Elapsed Time	3.126333333
	L1 DCM	15735824.3
	L2 DCM	32568592.7
1400 x 1400	Elapsed Time	8.868666667
	L1 DCM	44048338
	L2 DCM	88682901
1800 x 1800	Elapsed Time	16.6696667
	L1 DCM	95294913.3
	L2 DCM	186213881
2200 x 2200	Elapsed Time	30.9646667
	L1 DCM	262559120
	L2 DCM	344415443
2600 x 2600	Elapsed Time	55.2616667
	L1 DCM	546789905
	L2 DCM	562620346
3000 x 3000	Elapsed Time	103.056667
	L1 DCM	842652118
	L2 DCM	867881730
4096 x 4096	Elapsed Time	257.947
	L1 DCM	2159574853
	L2 DCM	2300220803
6144 x 6144	Elapsed Time	815.343333
	L1 DCM	7307878179
	L2 DCM	7804295098
8192 x 8192	Elapsed Time	1814.0385
	L1 DCM	1.7214E+10
	L2 DCM	1.9174E+10
10240 x 10240	Elapsed Time	-
	L1 DCM	-
	L2 DCM	-